Remori — Backend Study Guide

- **For:** the Remori backend + AI lead, who has finished the freeCodeCamp course "*Python API Development – Comprehensive Course for Beginners*" (Sanjeev Thiyagarajan, ~19 h, 2021).
- **Purpose:** follow the course's own structure, part by part, and list what the Remori backend needs that each part doesn't teach. Then cover the topics the course doesn't touch at all.
- **Scope:** backend and systems engineering only. The AI side (models, RAG, STT, vision, evaluation) is already covered by the reader's AI background.
- **Companion:** the backend conventions decided here are part of `REMORI_SPEC.md` §5.1.

How to read each part:

- ✅ **Course covered:** what you already know.
- 🔄 **Update:** the course used a 2021 version, so switch to the current one.
- ➕ **Missing for Remori:** what to add, and where Remori uses it.
- 🔎 **Study / search:** exact search terms and resources.

Summary: the two biggest gaps

1. **Async, message queues + background jobs (Celery + Redis).** Every cloud AI step in Remori (transcription, smart review, OCR, clustering, retention, summaries) runs as a background job. The course never leaves the request/response cycle. → Part B.
2. **Layered architecture (the "MVC" question).** The course writes all logic directly in route functions. Remori needs the same logic reachable from API routes **and** background workers, so it needs routers → services → models. → Part 6 and Part C.

Everything else is a small extension of what the course taught.

Part A — Following the course structure

Part 1 · Setup & Python environment (0:00 – 0:36)

✅ **Course covered:** Python install, VS Code, virtual environments, `pip`.

➕ **Missing for Remori:** nothing blocking. Skip the Mac install chapters.

Part 2 · FastAPI fundamentals (0:36 – 2:21)

Path operations, HTTP requests, Pydantic schema validation, CRUD, Postman, status codes, automatic docs.

✅ **Course covered:** routes, request bodies, Pydantic validation, CRUD, status codes, `/docs`.

🔄 **Update: Pydantic v1 → v2**

Course (v1)	Use now (v2)
`class Config: orm_mode = True`	`model_config = ConfigDict(from_attributes=True)`
`.dict()`	`.model_dump()`
`.parse_obj()`	`.model_validate()`

Course (v1)	Use now (v2)
`@validator`	`@field_validator`

➕ Missing for Remori

Topic	Where Remori uses it
`UploadFile` + `Form` (multipart/form-data)	`POST /patients/{id}/people`: 1–5 photos + name + relationship
Batch endpoints (a list in one request, partial-success response)	`POST /patients/{id}/events:batch`
One error shape for the whole API, via custom exception handlers	Spec §7.5: `{"error": {"code", "message"}}`

🔎 Study / search

- Pydantic docs → "*Migration Guide*" (v1 → v2)
- FastAPI docs → "*Request Files*", "*Request Forms and Files*", "*Handling Errors*"
- Search: `fastapi custom exception handler`, `fastapi UploadFile multiple files`

Part 3 · Databases & SQL (2:24 – 3:53)

Postgres install, schema & tables, pgAdmin, SELECT/WHERE/operators/LIKE/ORDER/LIMIT, modifying data.

✅ **Course covered:** tables, basic types, queries, filtering, ordering, pagination.

➕ Missing for Remori

Topic	Where Remori uses it
UUID primary keys (`gen_random_uuid()`, client-generated UUIDs)	Every table. Glasses and phone create IDs offline
Postgres ENUM types	`role`, `event_type`, `status`, …
JSONB columns + querying them	`events.metadata`, `media_assets.ai_tags`, `extracted_fields`
`TIMESTAMPTZ` vs `TIMESTAMP` (always store UTC)	Every timestamp
`UNIQUE` **constraints**, composite unique	`(caregiver_id, patient_id)`, `(patient_id, summary_date)`
`INSERT … ON CONFLICT DO NOTHING / DO UPDATE` (upsert)	Idempotent batch sync: the phone retries, and nothing duplicates
Soft delete (`deleted_at`)	Delta sync must tell the phone what was deleted
pgvector extension: `VECTOR(n)`, cosine distance `<=>`, **HNSW index**	Face embeddings (512-d), RAG embeddings (1536-d)

🔎 Study / search

- PostgreSQL docs → "*INSERT … ON CONFLICT*", "*JSON Types*", "*Date/Time Types*"
- GitHub → **pgvector/pgvector** README (operators, HNSW vs IVFFlat)
- Search: `postgres upsert on conflict`, `postgres timestamptz vs timestamp`, `soft delete pattern`, `pgvector hnsw index`

Part 4 · Python ↔ database and the ORM (3:53 – 5:50)

psycopg2, raw CRUD, ORM intro, SQLAlchemy setup and CRUD, Pydantic vs ORM models.

✅ **Course covered:** SQLAlchemy models, sessions, CRUD.

🔄 **Update: SQLAlchemy 1.x → 2.0 style**

Course (1.x)	Use now (2.0)
`db.query(Post).filter(...).first()`	`db.scalars(select(Post).where(...)).first()`
`Column(Integer, primary_key=True)`	`id: Mapped[uuid.UUID] = mapped_column(primary_key=True)`
`declarative_base()`	`class Base(DeclarativeBase)`

➕ **Missing for Remori**

Topic	Where Remori uses it
SQLAlchemy 2.0 typed models (`Mapped[]`, `mapped_column`)	All models
Upserts in SQLAlchemy: `postgresql.insert(...).on_conflict_do_nothing()`	`events:batch`, `reminder-logs:batch`
`pgvector.sqlalchemy.Vector` column + `.cosine_distance()` in queries	Face matching, RAG retrieval
Combining a SQL filter with vector ranking in **one** query	RAG: time window **then** similarity
Sync SQLAlchemy everywhere (spec §5.1)	Celery tasks are sync, so services stay sync and are shared by routes + tasks

🔎 **Study / search**

- SQLAlchemy docs → **"SQLAlchemy Unified Tutorial"** (the 2.0 way), "*PostgreSQL dialect → INSERT…ON CONFLICT*"
- GitHub → **pgvector/pgvector-python** README (SQLAlchemy section)
- Search: `sqlalchemy 2.0 mapped_column tutorial`, `sqlalchemy on_conflict_do_nothing`, `pgvector sqlalchemy cosine_distance`

Part 5 · Users, hashing, JWT auth (5:50 – 7:42)

Users table, registration, password hashing, JWT basics, login, OAuth2PasswordRequestForm, protected routes, expired tokens.

✅ **Course covered:** registration, hashing, access tokens, protecting routes.

🔄 **Update: libraries**

Course	Use now	Why
`python-jose`	**PyJWT**	The FastAPI docs moved to it, and jose is poorly maintained
`passlib`	**pwdlib** (or `bcrypt` directly)	passlib is unmaintained and breaks with new bcrypt

➕ **Missing for Remori**

Topic	Where Remori uses it
Refresh tokens (long-lived, stored hashed, rotated) + **revocation** / logout	Caregiver sessions: `/auth/refresh`, `/auth/logout`, `refresh_tokens` table

Topic	Where Remori uses it
Short access-token lifetime (15 min)	Spec §7.2
A second kind of principal: the device token (not a user password)	Patient's phone auto-login. `devices.token_hash`
Roles (`patient` / `caregiver`) + permission levels (`viewer` / `manager`)	`caregiver_patient_links.permission`

🔎 **Study / search**

- FastAPI docs → "*OAuth2 with Password (and hashing), Bearer with JWT tokens*" (current version, uses PyJWT)
- Search: `refresh token rotation`, `jwt refresh token revocation database`, `api key authentication fastapi`, `OWASP authentication cheat sheet`

Part 6 · Routers (6:17 – 6:32) → Layered architecture (the "MVC" gap)

FastAPI routers, prefixes, tags.

✅ **Course covered:** splitting routes into `APIRouter` files.

➕ **Missing for Remori: separating HTTP from logic from data access**

MVC vs. what an API backend uses. MVC comes from server-rendered apps (Django, Rails, Laravel, Spring MVC), where the "View" is an HTML page. Remori returns JSON to Flutter, so there's no View in that sense. API backends apply the same separation of concerns as a **layered architecture**:

MVC	FastAPI layer	Job
Controller	**Routers**	HTTP only: parse the request, call a service, return the response
(was inside Controller)	**Services**	Business logic, rules, orchestration, transactions
Model	**Repositories** (optional) + **ORM models**	Database access
View	**Pydantic schemas**	Request/response JSON shape
—	**Dependencies** (`Depends`)	Wiring: DB session, current user, authorization

Why it matters in Remori: "enroll a person" is called from an API route **and** from the unknown-face *promote* flow. "Transcribe a segment" runs in a worker. If logic lives inside route functions (like in the course), it gets duplicated. With services, **routes and workers call the same function**, and you can test it without HTTP.

Remori `backend-ai` layout (also in spec §5.1):

```
backend-ai/app/
├── api/v1/          # routers: people.py, events.py, reminders.py …  (thin)
├── services/        # people_service.py, sync_service.py, reminder_service.py (logic)
├── repositories/    # optional: shared complex queries (e.g. pgvector search)
├── models/          # SQLAlchemy ORM tables
├── schemas/         # Pydantic request/response
├── workers/         # Celery app + tasks: call the SAME services as the routers
├── core/            # config (pydantic-settings), security, db session, errors, logging
└── deps.py          # get_db, get_current_user, get_device, require_patient_link
```

Don't over-engineer: routers → services → models/schemas is enough. Add a repository only where several services share a complex query.

🔎 **Study / search**

- GitHub → **zhanymkanov/fastapi-best-practices** (read first)
- GitHub → **fastapi/full-stack-fastapi-template** (the FastAPI author's layout)
- Book → **"Architecture Patterns with Python" (Cosmic Python)**, Percival & Gregory, free at cosmicpython.com. **Chapters 1–6** (Domain model, Repository, Abstractions, Service layer, TDD, Unit of Work). Skip the event-driven chapters for now.
- Search: `service layer pattern`, `repository pattern python`, `fastapi dependency injection Depends`

Part 7 · Relationships & ownership (7:50 – 8:38)

Foreign keys, SQLAlchemy relationships, assigning owner, "delete/update only your own", "only retrieve the logged-in user's".

✅ **Course covered:** FKs, relationships, owner checks inside routes.

➕ **Missing for Remori: authorization as a reusable dependency**

- In the course, "only your own" is an `if` inside each route. Remori has **many** routes under `/patients/{id}/…`. Each must check: *is the caller a caregiver linked to this patient, or the device token of this patient?* Write that once as a dependency (`require_patient_link`) and attach it to every patient router. Forgetting it on one route is an **IDOR** vulnerability.
- Permission levels: a `viewer` caregiver can read but not edit reminders.

🔎 **Study / search**

- Search: `fastapi dependency authorization per resource`, `IDOR vulnerability`, `OWASP broken object level authorization` (OWASP API Security Top 10, item API1)

Part 8 · Query params, env vars, votes, joins (8:38 – 10:28)

Query parameters, cleanup, environment variables, votes table, SQL joins, joins in SQLAlchemy.

✅ **Course covered:** query params, `.env`, joins.

🔄 **Update:** use `pydantic-settings` (`BaseSettings` moved out of Pydantic v2 into its own package).

➕ **Missing for Remori**

Topic	Where Remori uses it
`pydantic-settings` with separate dev / test / prod settings	`core/config.py`
Keeping **secrets** out of git (AI API keys, AWS keys, JWT secret)	`.env` + GitHub Secrets + AWS IAM roles

🔎 **Study / search:** Pydantic docs → "*Settings Management*"

Part 9 · Alembic & CORS (10:30 – 11:23)

What is a migration tool, Alembic setup, CORS.

✅ **Course covered:** Alembic basics, autogenerate, CORS.

➕ **Missing for Remori**

Topic	Why
Enabling an extension in a migration: `CREATE EXTENSION IF NOT EXISTS vector`	pgvector must exist before vector columns
Postgres ENUMs in Alembic (autogenerate doesn't handle enum changes well)	Adding a new `event_type` later needs a manual migration
Reviewing autogenerated migrations by hand	Autogenerate misses some changes, and gets others wrong

🔎 **Study / search:** `alembic postgres enum add value`, `alembic create extension`, `alembic autogenerate limitations`

Part 10 · Git, GitHub, Heroku (11:23 – 12:05)

✅ **Course covered:** Git basics.

➕ **Skip Heroku** (its free tier is gone). Remori deploys to **AWS**. See Part 11.

Part 11 · Ubuntu VM deployment (12:05 – 13:23)

Ubuntu VM, Postgres setup, Gunicorn, systemd, NGINX, domain, SSL, firewall.

✅ **Course covered:** this transfers directly to an **AWS EC2** instance.

➕ **Missing for Remori**

Topic	Where Remori uses it
EC2 (instance types, key pairs, Elastic IP)	Runs the Docker Compose stack
Security groups (AWS's firewall)	Only 443/80 public, Postgres and Redis private
S3 buckets, private by default	Media + audio (Part E)
IAM roles for EC2 (no access keys in `.env`)	The server talks to S3 securely
(Option) **RDS** for Postgres	vs. Postgres in a container
Free-tier / credits limits	AWS changed new-account free tier in 2025, so check your account

🔎 **Study / search:** AWS docs → "*Get started with Amazon EC2*", "*IAM roles for Amazon EC2*", "*Security groups*". Search: `deploy fastapi docker compose ec2`

Part 12 · Docker (13:26 – 14:08)

Dockerfile, Docker Compose, Postgres container, bind mounts, Docker Hub, prod vs dev.

✅ **Course covered:** Dockerfile, Compose, Postgres container.

➕ **Missing for Remori:** Compose with **more services**: `api`, `worker` (Celery, same image, different command), `beat` (Celery Beat, exactly one), `flower` (monitoring), `redis` (with AOF persistence enabled), `postgres` (with the `pgvector/pgvector` image). Health checks, plus `depends_on` with `condition: service_healthy`.

🔎 **Study / search:** `docker compose healthcheck depends_on condition`, `pgvector docker image`, `redis docker persistence appendonly`

Part 13 · Testing with pytest (14:14 – 17:34)

First tests, parametrize, classes, fixtures, TestClient, test database, conftest.

✅ **Course covered:** pytest, fixtures, TestClient, test database.

➕ **Missing for Remori**

Topic	Why
Mocking external HTTP APIs with `respx` (for `httpx`)	Test STT / LLM / vision code without paying for or depending on real APIs
Testing services directly (no HTTP)	Possible once the layered architecture is in place
Testing Celery tasks: call them directly or in eager mode (`task_always_eager`) for unit tests, one integration test with a real worker, and **idempotency tests** (run a task twice, assert one result)	Core guarantee of the job system
Testing authorization (caregiver A must get 403/404 on patient B)	Catches IDOR

🔎 **Study / search:** `respx httpx mock pytest`, `pytest-asyncio`, `testing idempotency`

Part 14 · CI/CD with GitHub Actions (17:34 – end)

Jobs, Python setup, env vars, secrets, test database, Docker build, deploy.

✅ **Course covered:** a full pipeline for one project.

➕ **Missing for Remori:** it's a **monorepo** (`flutter/`, `backend-ai/`, `ml/`, `hardware/`), so use **path filters**: a Flutter change runs only Flutter jobs, and a backend change runs only backend tests. Also add a service container for Postgres + pgvector and Redis in CI.

🔎 **Study / search:** `github actions paths filter monorepo`, `github actions service containers postgres redis`

Part B — Not in the course: Async, message queues & background jobs (the biggest gap)

Remori's choice (spec D19): Celery + Redis as the broker + Celery Beat for cron + Flower for monitoring.

Three layers, three things to learn

```
Your code:        transcribe.delay(seg_42)            <- "run this function later"
Task queue:       Celery                              <- workers, retries, routing, cron (Beat)
Message broker:   Redis  (alternatives: RabbitMQ, SQS) <- stores and delivers the messages
```

- A **message queue (broker)** only moves messages: a producer puts one in, a consumer takes it out and **acknowledges** it. It knows nothing about functions or retries.
- A **task queue** is a framework that turns those messages into function calls, and adds workers, retries, scheduling and results.
- **Async (asyncio)** is a third, separate thing: one process doing many things while waiting on I/O. It's how FastAPI serves many requests at once.

Async vs. background jobs

	Async / concurrency (asyncio)	**Background job queue (Celery + Redis)**
Problem	One process does many things **while waiting** on I/O	Move slow / unreliable work **out of the request entirely**
Where work runs	Inside the request. The phone waits	In a separate **worker** process. The phone gets 200 in ms
Survives a crash?	No	Yes: the message is stored in Redis
Retries / scheduling	No	Yes: retries with backoff + Celery Beat

```
Phone --POST /uploads/{id}/complete--> API --commit, then transcribe.delay(seg_42)--> Redis
   <-- 200 OK in ~20 ms --------------+                                                  |
                                                                                        v
                                    Celery worker (separate process) takes the message, calls
                                    Whisper (5-30 s), writes the transcript, deletes the audio,
                                    then acks. Retries with backoff if Whisper fails.
```

Remori tasks: transcribe audio + delete it · smart review + OCR · render + embed events · cluster unknown faces · retention expiry (Beat, hourly) · daily summaries (Beat) · alert rules (Beat: missed meds, device offline, low battery).

Why not FastAPI's `BackgroundTasks`? It runs in the **same process** after the response. A restart loses the work, there are no retries and no scheduling, and it ties up API workers.

Message queues you should know

Broker	**What it is**	**In Remori?**
Redis (lists / streams)	In-memory store that also works as a simple broker	✅ The Celery broker

Broker	What it is	In Remori?
RabbitMQ	A dedicated broker (AMQP): exchanges, routing keys, acks, dead-letter queues	Not needed, but the classic one to learn
AWS SQS	Managed queue, nothing to run yourself	Possible later
Kafka	Not really a queue: a **distributed event log**. Messages are kept, can be replayed, and are read by consumer groups at very high throughput	❌ Overkill: built for event streaming at scale

Concepts that apply to every broker: point-to-point vs. **publish/subscribe** · **acknowledgements** (ack after the work, not before) · **at-least-once delivery** · **dead-letter queues** · **backpressure** · **visibility timeout** (how long a message stays hidden before it's redelivered).

A queue you already designed: glasses → phone → cloud. Each tier holds items until the next one acknowledges them. That's the **transactional outbox pattern** (the phone's drift outbox).

Celery vs. ARQ (why Celery)

	Celery	**ARQ**
Popularity	Industry standard in Python, and what jobs ask for	Small, niche
Brokers	Redis, RabbitMQ, SQS	Redis only
Async	Tasks are **sync**. Concurrency comes from the worker pool (prefork, threads, gevent)	asyncio-native
Cron	**Celery Beat** (separate process)	Built in
Extras	`chain` / `group` / `chord`, rate limits, queue routing, **Flower** dashboard	Minimal

Remori's tasks mostly **wait on APIs**, so sync tasks on a **threads** pool work well. Write them as normal functions with a sync `httpx.Client`.

"Safe to run twice" (idempotency)

Delivery is **at-least-once**, not exactly-once. If the worker dies after Whisper answered but before the ack, Redis redelivers the message and the task runs again. Design every task so a second run is harmless:

- **Check state first:** if the segment is already `transcribed`, stop.
- **Deterministic IDs + upserts:** the transcript event's ID is derived from the segment ID, so a re-run overwrites instead of duplicating.
- **No duplicate concurrent runs:** Celery does **not** deduplicate by `task_id`. Use a short **Redis lock** (`SET key NX EX ttl`, key `transcribe:seg_42`) so two copies never run at the same time.
- **Enqueue after commit:** send the task only after the DB transaction commits, otherwise the worker may look for a row that doesn't exist yet.

Study plan

Step 1: async in FastAPI (1–2 days)

- FastAPI docs → **"Concurrency and async / await"** (when to use `def` vs `async def` routes)
- Real Python → **"Async IO in Python: A Complete Walkthrough"**
- Video → **"import asyncio"** series by **Łukasz Langa** (EdgeDB YouTube channel)
- Make sure you understand: the event loop · `await` · `asyncio.gather` · why blocking code (`time.sleep`, `requests`, CPU-heavy inference) freezes the loop · why a `def` route runs in a threadpool · `httpx` sync vs async clients

Step 2: message-queue concepts (2 days)

- **RabbitMQ official tutorials 1–5** (Python). They're short, and they teach queues, work queues, acks, pub/sub and routing, the concepts behind every broker
- Search: `pub sub vs message queue`, `at-least-once vs exactly-once delivery`, `dead letter queue`, `kafka vs rabbitmq`, `exponential backoff with jitter` (AWS Architecture Blog), `transactional outbox pattern`
- Read: Stripe blog → **"Designing robust and predictable APIs with idempotency"**
- Read: Brandur Leach → **"Implementing Stripe-like Idempotency Keys in Postgres"** (directly relevant to `events:batch`)

Step 3: Redis basics (1 day)

- Search: `Redis data types tutorial`, `Redis persistence RDB vs AOF`, `Redis SET NX EX lock`, `Redis docker compose`
- You need: what Redis is, running it in Docker, persistence on (AOF), and simple locks

Step 4: Celery (3–4 days)

- Celery docs, in this order:
  - **"First Steps with Celery"** and **"Next Steps"**
  - **"Tasks"**: `bind=True`, `autoretry_for`, `retry_backoff`, `retry_jitter`, `max_retries`, **`acks_late`**, `ignore_result`
  - **"Routing Tasks"**: separate queues (`ai_io`, `cpu`, `default`)
  - **"Periodic Tasks"**: **Celery Beat**, `crontab` schedules, and why only one Beat may run
  - **"Workers Guide"**: pools (`prefork` vs `threads`), `--concurrency`, `worker_prefetch_multiplier`
  - **"Using Redis"** (broker page): **`visibility_timeout`** and its caveats
  - **"Signals"**: `worker_process_init` to load models once per process
  - **"Canvas"**: `chain` / `group` / `chord` (good to know, rarely needed in Remori)
- **Flower** docs: run the dashboard, inspect tasks
- Search: `celery fastapi docker compose`, `celery acks_late reject_on_worker_lost`, `celery retry exponential backoff`, `celery redis visibility timeout`, `celery task idempotent`, `celery eager mode testing`

Related, same week:

Topic	Where Remori uses it	Study / search
Serving AI inside a backend: load ONNX models once per process, keep inference off the request path, timeouts on every external call	Face pipeline at enrollment, all AI API calls	`onnxruntime inference session reuse`, `celery worker_process_init`
Push notifications: `firebase-admin` (FCM)	Caregiver alerts	Firebase docs → "*Send messages to specific devices*" (Admin SDK, Python)

Step 5: practice project (before any Remori code)

1. FastAPI endpoint `POST /tasks` writes a row, **commits**, then calls `my_task.delay(id)` and returns immediately.
2. A Celery task calls a fake slow API (`time.sleep(5)`) that **randomly fails** with a `TransientError`.
3. Add `autoretry_for` + `retry_backoff` + `retry_jitter`, and watch it recover in **Flower**.
4. Turn on `acks_late`, **kill the worker mid-task**, restart it, and verify the task is redelivered and completes **without duplicating its result** (idempotency proven).
5. Enqueue the same task twice at once and show the **Redis lock** lets only one run.
6. Add a **Celery Beat** schedule that runs every minute.

7. Route one task to a second queue and run a second worker with a different pool.
8. Run it all with Docker Compose: `api` + `worker` + `beat` + `flower` + `redis` + `postgres`.

If this toy project works, you have everything Remori's cloud workers need.

Part C — Not in the course: production backend essentials

C1. Offline-first sync

Topic	Where Remori uses it	Study / search
Client-generated UUIDs + idempotent inserts	Glasses/phone create records offline	`client generated uuid offline sync`
Delta sync: `GET /sync?since=T` using an `updated_at` cursor + `deleted_at` tombstones	Phone pulls people, reminders, places	`delta sync updated_at cursor`, `tombstone soft delete sync`
Conflict rules	Reminders: `completed` always wins, else latest timestamp (spec §7.1)	`last write wins conflict resolution`, `offline first sync patterns`

C2. Time & recurring schedules

Topic	Where Remori uses it	Study / search
`zoneinfo`, aware datetimes, always storing UTC	Every timestamp, patient timezone `Africa/Cairo`	Python docs → *"zoneinfo"*. Search `python aware vs naive datetime`
DST (Egypt observes it again since 2023)	A 09:00 reminder must stay 09:00 local	`daylight saving time scheduling bugs`
RRULE with `python-dateutil`	Expanding reminders into 48 h of occurrences	dateutil docs → *"rrule"*. Search `RFC 5545 RRULE examples`

C3. Files & object storage (S3)

Topic	Where Remori uses it	Study / search
boto3 basics	All media + audio	boto3 docs → *"S3"*
Presigned PUT / GET URLs, short expiry	Phone uploads directly to S3. Caregiver views through signed links	boto3 docs → *"Presigned URLs"*
Private buckets, never public URLs	Photos of people + audio are sensitive	`s3 block public access`
Two-step upload: request URL → upload → `complete`	`/uploads` → S3 → `/uploads/{id}/complete` → enqueue job	`s3 presigned upload confirm callback pattern`

C4. Error handling & logging

Topic	Where Remori uses it	Study / search
Domain exceptions raised in services (`PersonNotFound`), mapped **once** to the spec's error shape by exception handlers	Spec §7.5	FastAPI docs → *"Handling Errors"*
Structured logging (JSON logs) with a **request ID** that's passed into worker jobs	Trace one upload through API → Redis → worker	`structlog fastapi`, `request id middleware fastapi`, `correlation id logging`

Topic	Where Remori uses it	Study / search
Audit log for sensitive actions	`audit_log` table: who viewed or deleted which photo	`audit logging best practices`

C5. As team lead (light)

Topic	Why	Study / search
BLE GATT basics: services, characteristics, read/write/notify, MTU, bonding	Review the hardware and Flutter teams' work against the §7.1 contract	`BLE GATT explained`, `BLE characteristics notify vs indicate`, `BLE MTU throughput`
Monorepo CI	See Part 14	—

Part D — Recommended order

When	Study	Unlocks (spec §12 build step)
Days 1–2	Part 2/4/5 **updates** (Pydantic v2, SQLAlchemy 2.0, PyJWT, pwdlib)	—
Week 1–2	**Part B** (async → message-queue concepts → Redis → Celery) + practice project	Steps 9–10 (workers)
Week 1–2	**Part 6** layered architecture + **Part 7** authorization dependency	Step 2 (FastAPI skeleton)
Week 2	**Part 3/4** UUID, ENUM, JSONB, upserts, pgvector · **Part 5** refresh + device tokens · **C1** sync · **C2** time	Steps 2–3 (schema, core API)
Week 3	**C3** S3 · **C4** errors + logging · **Part 13** respx, worker + idempotency tests	Steps 3, 10
Later	**Part 11** AWS · **Part 12** Compose · **Part 14** monorepo CI · **C5** BLE	Deploy, CI, integration reviews

One-line summary: Message queues + Celery background jobs, and layered architecture, are the two real gaps. Everything else extends what the course already taught you.